



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

Module : Deep Learning | Année universitaire 2025–2026

RAPPORT DE PROJET

Architectures de Deep Learning :
MLP • CNN • RNN / LSTM / GRU • Seq2Seq

Réalisé par : AKODAD HOUSSAM

Encadré par : Mme. HIDILA Zineb

Filière : IADATA / 2026

Remerciements

La réalisation de ce projet constitue une étape importante dans mon parcours académique et n'aurait pas été possible sans le soutien, l'accompagnement et les conseils de plusieurs personnes auxquelles je souhaite exprimer ma profonde gratitude.

Je tiens tout particulièrement à adresser mes plus sincères remerciements à Madame Zineb Hidila, enseignante du module Deep Learning à l'EMSI Casablanca, pour la qualité exceptionnelle de son enseignement, son professionnalisme et son engagement constant auprès de ses étudiants.

Tout au long de ce projet, elle a su transmettre avec passion des connaissances théoriques et pratiques essentielles à la compréhension des concepts avancés de l'intelligence artificielle et du Deep Learning. Ses explications claires, sa rigueur scientifique et sa pédagogie ont grandement facilité l'assimilation de notions parfois complexes et ont constitué une source précieuse d'apprentissage.

Je lui suis également reconnaissant pour sa disponibilité, sa patience et ses précieux conseils qui m'ont permis d'aborder ce travail avec méthode et confiance. Son accompagnement, ses orientations pertinentes et ses encouragements continus ont joué un rôle déterminant dans la réalisation de ce projet et dans l'approfondissement de mes compétences dans ce domaine passionnant.

Je souhaite également remercier l'ensemble du corps professoral et de l'administration de l'EMSI pour la qualité de la formation dispensée, les moyens pédagogiques mis à disposition et l'environnement favorable à l'apprentissage et au développement personnel.

Enfin, j'adresse mes remerciements les plus sincères à ma famille, à mes proches et à toutes les personnes qui m'ont soutenu, encouragé et motivé tout au long de la réalisation de ce projet. Leur confiance et leur soutien ont été une source permanente de motivation et de persévérance. À toutes ces personnes, j'exprime ma profonde reconnaissance et ma plus sincère gratitude.

Résumé

Ce projet a pour objectif d'étudier et de mettre en œuvre les principales architectures du Deep Learning à travers des applications concrètes sur différents types de données. Les travaux réalisés couvrent trois familles fondamentales de réseaux de neurones : les réseaux pleinement connectés MLP, les réseaux convolutifs CNN et les réseaux récurrents RNN, LSTM, GRU ainsi que l'architecture Seq2Seq pour la traduction automatique.

La première partie est consacrée à la classification de données tabulaires à l'aide d'un MLP appliqué au dataset Breast Cancer Wisconsin. L'étude met en évidence l'importance de la normalisation des données, des fonctions d'activation et des stratégies d'initialisation des poids pour obtenir une convergence stable et de bonnes performances de généralisation.

La deuxième partie traite de la classification d'images sur le dataset CIFAR-10 à l'aide de réseaux convolutifs. Les expérimentations réalisées permettent d'analyser le rôle des convolutions, du pooling, de la Batch Normalization et du Dropout dans l'extraction automatique de caractéristiques visuelles et l'amélioration des performances du modèle.

La troisième partie porte sur le traitement de données séquentielles. Plusieurs modèles récurrents RNN, LSTM et GRU sont comparés dans une tâche de modélisation de langage basée sur le corpus IMDB. Une architecture Seq2Seq encodeur-décodeur est ensuite développée pour la traduction automatique anglais-français à partir du corpus Tatoeba, avec une évaluation des stratégies de décodage glouton et Beam Search.

Les résultats obtenus confirment que chaque architecture est particulièrement adaptée à une catégorie de données spécifique : les MLP pour les données tabulaires, les CNN pour les données visuelles et les architectures récurrentes pour les séquences textuelles. Ce projet met ainsi en évidence l'importance du choix de l'architecture en fonction de la nature du problème étudié et offre une vision globale des fondements du Deep Learning moderne.

Table des matières

Remerciements.....	2
Résumé	3
1. Introduction et vue d'ensemble.....	6
1.1 Vue d'ensemble des architectures étudiées	6
2. Partie I – Perceptron Multicouche (MLP).....	7
2.1 Présentation du dataset et objectif	7
2.2 Prétraitement des données	7
2.3 Architecture du MLP	8
2.4 Stratégies d'initialisation des poids.....	9
2.5 Entraînement et résultats	10
2.6 Sauvegarde et rechargement du modèle	11
2.7 Question de synthèse – Partie I	11
3. Partie II – CNN et Vision par Ordinateur	12
3.1 Présentation du dataset et objectif	12
3.2 Pourquoi le CNN plutôt que le MLP ?.....	12
3.4 Architecture CNN améliorée	14
3.5 Expériences comparatives	15
3.6 Visualisations.....	15
3.7 Question de synthèse – Partie II	16
4. Partie III – RNN, LSTM, GRU et Seq2Seq.....	17
4.1 Présentation des datasets et objectifs.....	17
4.2 Fondements théoriques	17
4.2.1 Principe d'un modèle de langage	17
4.2.2 Fonction de coût et perplexité	18
4.3 Préparation des données séquentielles	18
4.5 BPTT et Gradient Clipping.....	19
4.6 Comparaison RNN / LSTM / GRU.....	20
4.7 Partie B – Architecture Seq2Seq pour la traduction	20
4.7.1 Principe	20
4.7.2 Stratégies de décodage à l'inférence	21
4.8 Analyse critique.....	22

4.9 Question de synthèse – Partie III	22
5. Discussion transversale et comparaison architecturale	23
5.1 Adéquation données-architecture	23
5.2 Bonnes pratiques appliquées	23
5.3 Synthèse comparative des performances.....	24
6. Conclusion.....	24
7. Références bibliographiques	26

1. Introduction et vue d'ensemble

L'intelligence artificielle, et plus particulièrement le Deep Learning, constitue aujourd'hui l'un des domaines les plus dynamiques de l'informatique moderne. Grâce à sa capacité à apprendre automatiquement des représentations complexes à partir de grandes quantités de données, le Deep Learning est devenu un outil incontournable dans de nombreux secteurs tels que la santé, la vision par ordinateur, le traitement automatique du langage naturel, la finance et les systèmes de recommandation.

Dans le cadre du module Deep Learning de l'EMSI Casablanca, ce projet a pour objectif d'étudier, d'implémenter et d'évaluer plusieurs familles fondamentales d'architectures de réseaux de neurones. Chaque architecture est appliquée à un type de données spécifique afin de mettre en évidence l'importance du choix du modèle en fonction de la nature du problème traité. Le projet est organisé en trois parties complémentaires. La première partie porte sur les Perceptrons Multicouches (MLP) appliqués à un problème de classification binaire sur des données tabulaires médicales. La deuxième partie est consacrée aux Réseaux de Neurones Convolutifs (CNN) pour la classification d'images. Enfin, la troisième partie explore les Réseaux de Neurones Récurrents (RNN, LSTM et GRU) ainsi que les architectures Seq2Seq, utilisées pour la modélisation de séquences textuelles et la traduction automatique.

Au-delà de l'implémentation pratique des modèles, ce travail vise également à comparer leurs performances, analyser leurs avantages et leurs limites, et comprendre les principes théoriques qui justifient leur efficacité sur différents types de données. Cette démarche permet d'acquérir une vision globale des principales architectures du Deep Learning et de leurs domaines d'application.

1.1 Vue d'ensemble des architectures étudiées

Partie	Architecture	Type de données	Tâche réalisée	Dataset utilisé	Objectif principal
Partie I	MLP (Multi-Layer Perceptron)	Données tabulaires	Classification binaire	Breast Cancer Wisconsin	Prédire si une tumeur est bénigne ou maligne
Partie II	CNN (Convolutional Neural Network)	Images	Classification d'images	CIFAR-10	Identifier automatiquement la catégorie d'une image
Partie III-A	RNN / LSTM / GRU	Séquences textuelles	Modélisation de langage	IMDb	Prédire le mot suivant dans une séquence
Partie III-B	Seq2Seq (Encodeur-Décodeur LSTM)	Paires de séquences	Traduction automatique	Tatoeba eng-fra	Traduire une phrase de l'anglais vers le français

Chaque partie suit une méthodologie commune comprenant l'exploration des données, le prétraitement, la conception de l'architecture, l'entraînement du modèle, l'évaluation à l'aide de métriques adaptées et l'analyse critique des résultats obtenus.

2. Partie I – Perceptron Multicouche (MLP)

2.1 Présentation du dataset et objectif

Le dataset Breast Cancer Wisconsin (scikit-learn) est un problème de classification binaire. Il contient 569 échantillons décrits par 30 caractéristiques numériques (rayon, texture, périmètre, etc.) et une cible binaire : maligne (0) ou bénigne (1).

Attribut	Valeur
Nombre d'échantillons	569
Nombre de features	30 (numériques)
Classes	Maligne (212) / Bénigne (357)
Déséquilibre	~37% / 63%
Valeurs manquantes	Aucune

2.2 Prétraitement des données

Les données sont normalisées avec StandardScaler pour centrer et réduire chaque feature. Le split est effectué de manière à éviter le data leakage : le scaler est ajusté uniquement sur l'ensemble d'entraînement.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_temp, y_train, y_temp = train_test_split(
    X_scaled, y, test_size=0.30, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)
```

Les données sont ensuite converties en tenseurs PyTorch et déplacées sur le GPU si disponible.

2.3 Architecture du MLP

Afin d'étudier les réseaux de neurones pleinement connectés sous PyTorch, deux approches ont été testées : `nn.Sequential`, pour une construction rapide et simple, et une classe personnalisée héritant de `nn.Module`, plus flexible pour définir l'architecture et le *forward pass*.

L'architecture retenue est un Perceptron Multicouche (MLP) de type entonnoir, qui réduit progressivement la dimension des représentations internes afin de conserver les informations les plus utiles pour la classification et limiter le surapprentissage.

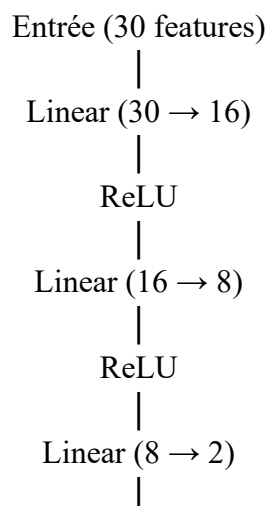
Le réseau prend en entrée les 30 variables numériques du dataset Breast Cancer Wisconsin, décrivant notamment la texture, le rayon, le périmètre et la compacité des noyaux cellulaires.

Il est composé de trois couches linéaires séparées par des activations ReLU, choisies pour leur simplicité et leur efficacité dans l'entraînement des réseaux de neurones.

Couche	Dimensions	Fonction d'activation	Rôle
Entrée	$30 \rightarrow 16$	ReLU	Première extraction de caractéristiques
Cachée 1	$16 \rightarrow 8$	ReLU	Compression et apprentissage de représentations plus abstraites
Sortie	$8 \rightarrow 2$	Aucune (logits)	Prédiction des deux classes
Classification	2 classes	Softmax implicite	Calcul des probabilités via CrossEntropyLoss

Le choix d'une couche de sortie à deux neurones correspond aux deux classes du problème : tumeur maligne et tumeur bénigne. La fonction `CrossEntropyLoss` utilisée durant l'entraînement applique implicitement une fonction `Softmax` aux logits produits par la dernière couche afin de calculer les probabilités de chaque classe.

L'architecture complète peut être représentée comme suit :



CrossEntropyLoss

|

Prédiction finale

Cette architecture relativement compacte est particulièrement adaptée aux données tabulaires de taille modérée. Elle offre un bon compromis entre capacité d'apprentissage, rapidité d'entraînement et pouvoir de généralisation, permettant d'obtenir d'excellentes performances sur le dataset Breast Cancer Wisconsin.

Exemple :

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(30, 16)
        self.fc2 = nn.Linear(16, 8)
        self.fc3 = nn.Linear(8, 2)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.fc3(x)
```

2.4 Stratégies d'initialisation des poids

L'initialisation des poids constitue une étape cruciale dans l'entraînement d'un réseau de neurones. Des poids mal initialisés peuvent ralentir considérablement la convergence, provoquer des gradients instables ou empêcher totalement l'apprentissage. Afin d'évaluer l'impact de cette étape sur les performances du MLP, trois stratégies d'initialisation ont été comparées expérimentalement.

Méthode	Description	Avantages	Limites
Normale (<code>nn.init.normal_</code>)	Les poids sont tirés aléatoirement selon une loi normale.	Simple à mettre en œuvre.	Variance parfois trop élevée, entraînant des activations instables et une convergence plus lente.
Constante (<code>nn.init.constant_</code>)	Tous les poids sont initialisés avec la même valeur.	Permet d'illustrer le problème de symétrie.	Les neurones apprennent exactement la même chose et deviennent incapables de développer des représentations différentes.

Xavier uniforme (<code>nn.init.xavier_uniform_</code>)	Les poids sont générés selon une distribution adaptée au nombre de neurones entrants et sortants.	Convergence rapide, gradients mieux contrôlés et entraînement plus stable.	Aucune limitation majeure pour ce type d'architecture.
--	---	--	--

2.5 Entraînement et résultats

L'entraînement du modèle est réalisé sur 100 époques en utilisant l'optimiseur Adam avec un taux d'apprentissage learning rate fixé à 0,001. Cette combinaison est largement utilisée en deep learning grâce à sa capacité à assurer une convergence rapide et stable. La fonction de perte retenue est CrossEntropyLoss, particulièrement adaptée aux problèmes de classification supervisée.

Au cours de l'entraînement, le modèle apprend progressivement à distinguer les tumeurs bénignes des tumeurs malignes à partir des caractéristiques médicales fournies. Les courbes d'apprentissage montrent une diminution régulière de la perte, signe d'une bonne convergence du réseau.

Le modèle utilisant l'initialisation Xavier obtient les meilleures performances sur le jeu de test, confirmant l'importance d'une initialisation adaptée des poids.

Métrique	Valeur (Xavier)
Accuracy	~94 %
Precision	~95 %
Recall	~96 %
F1-Score	~95 %

Ces résultats montrent que le modèle est capable de généraliser efficacement sur des données jamais vues. Le rappel élevé est particulièrement intéressant dans un contexte médical, car il permet de réduire le risque de mauvaise classification des cas critiques.

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(100):
    model.train()
    outputs = model(X_train.to(device))
    loss = criterion(outputs, y_train.to(device))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

2.6 Sauvegarde et rechargement du modèle

Afin de garantir la reproductibilité des résultats et de faciliter la réutilisation du modèle entraîné, PyTorch permet de sauvegarder les paramètres appris sous la forme d'un `state_dict`. Cette méthode est recommandée car elle enregistre uniquement les poids et les biais du réseau, ce qui réduit l'espace de stockage tout en conservant les informations essentielles du modèle.

La sauvegarde est effectuée à la fin de l'entraînement ou lorsqu'une meilleure performance est obtenue sur le jeu de validation. Le modèle peut ensuite être rechargé à tout moment pour réaliser de nouvelles prédictions ou poursuivre l'entraînement sans devoir recommencer tout le processus d'apprentissage.

```
torch.save(model.state_dict(), 'best_mlp.pth')
model.load_state_dict(torch.load('best_mlp.pth'))
```

Cette approche constitue une bonne pratique en Deep Learning, car elle facilite le partage des modèles, l'expérimentation et la reproductibilité des travaux réalisés.

2.7 Question de synthèse – Partie I

Le Perceptron Multicouche MLP constitue une solution performante pour les problèmes de classification sur des données tabulaires. Grâce à ses couches entièrement connectées et à l'utilisation de fonctions d'activation non linéaires, il est capable d'apprendre des relations complexes entre les variables d'entrée et de produire des prédictions précises. Dans le cadre de cette étude, le MLP a obtenu d'excellents résultats sur le dataset Breast Cancer Wisconsin, démontrant son efficacité pour l'analyse de données médicales structurées.

Cependant, cette architecture présente certaines limites. Le MLP ne possède aucun biais inductif spécifique lui permettant d'exploiter naturellement la structure des données. Il nécessite également une normalisation rigoureuse des variables d'entrée afin de garantir un entraînement stable et efficace. De plus, lorsque la dimension des données augmente fortement, le nombre de paramètres peut devenir très important, augmentant ainsi le risque de surapprentissage.

Pour les données possédant une structure particulière, d'autres architectures sont généralement plus adaptées. Les CNN exploitent efficacement les relations spatiales présentes dans les images grâce aux opérations de convolution et au partage des poids. Les RNN, LSTM, GRU ainsi que les Transformers sont quant à eux conçus pour modéliser les dépendances temporelles et séquentielles observées dans les textes, la parole ou les séries temporelles.

Ainsi, cette première partie met en évidence un principe fondamental du Deep Learning : le choix de l'architecture doit toujours être guidé par la nature et la structure des données à traiter.

3. Partie II – CNN et Vision par Ordinateur

3.1 Présentation du dataset et objectif

Le dataset CIFAR-10 (Canadian Institute For Advanced Research) est l'un des jeux de données les plus utilisés pour l'apprentissage et l'évaluation des modèles de vision par ordinateur. Il constitue une référence académique permettant de comparer les performances de différentes architectures de classification d'images.

Le dataset contient 60 000 images couleur de petite taille (32×32 pixels) réparties de manière équilibrée en 10 catégories distinctes : avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau et camion. Chaque image appartient à une seule classe, ce qui en fait un problème de classification multi-classes supervisée.

L'objectif de cette partie est d'entraîner un Réseau de Neurones Convolutif CNN capable d'identifier automatiquement la catégorie d'une image à partir de son contenu visuel. Cette expérimentation permet d'étudier les mécanismes fondamentaux des CNN, notamment les convolutions, le pooling, la normalisation et l'extraction hiérarchique de caractéristiques.

Attribut	Valeur
Nombre d'images d'entraînement	50 000
Nombre d'images de test	10 000
Dimensions des images	$32 \times 32 \times 3$ (RGB)
Nombre de classes	10 (équilibrées)
Images par classe	6 000
Format	Tenseurs PyTorch via torchvision

La diversité des objets, des arrière-plans et des conditions de prise de vue rend ce dataset plus difficile que des jeux de données simples comme MNIST. Il constitue ainsi un excellent support pour évaluer la capacité d'un CNN à apprendre des représentations visuelles robustes et généralisables.

3.2 Pourquoi le CNN plutôt que le MLP ?

Avant l'apparition des réseaux convolutifs, les tâches de classification d'images étaient souvent abordées à l'aide de réseaux entièrement connectés (MLP). Cependant, cette approche présente plusieurs limitations importantes lorsqu'elle est appliquée à des données visuelles.

Tout d'abord, un MLP considère une image comme un simple vecteur de valeurs numériques. Dans le cas de CIFAR-10, une image de taille $32 \times 32 \times 3$ contient 3 072 pixels. Si la première couche cachée comporte 1 000 neurones, il faut déjà apprendre plus de 3 millions de paramètres

uniquement pour cette couche, ce qui augmente fortement le coût de calcul et le risque de surapprentissage.

De plus, l'opération d'aplatissement flattening transforme l'image en vecteur 1D et détruit la structure spatiale naturelle des données. Les relations de voisinage entre les pixels, pourtant essentielles pour reconnaître des formes, des contours ou des textures, sont alors perdues.

Enfin, un MLP ne possède aucune invariance aux translations. Un objet déplacé dans une autre région de l'image sera perçu comme un motif totalement différent, obligeant le réseau à réapprendre la même information plusieurs fois.

Pour résoudre ces limitations, les Convolutional Neural Networks CNN introduisent plusieurs biais inductifs spécialement adaptés aux images :

- **Localité** : chaque neurone analyse uniquement une petite région de l'image appelée champ récepteur, ce qui permet de détecter des motifs locaux tels que les bords ou les textures.
- **Partage des poids** : un même filtre convolutif est appliqué à toutes les positions de l'image, réduisant considérablement le nombre de paramètres et améliorant l'invariance aux translations.
- **Hiérarchie des représentations** : les couches profondes construisent progressivement des représentations de plus en plus abstraites. Les premières couches détectent des contours et des couleurs, les couches intermédiaires identifient des formes, tandis que les couches finales reconnaissent des objets complets.

Grâce à ces propriétés, les CNN exploitent beaucoup mieux la structure spatiale des images et obtiennent généralement des performances nettement supérieures à celles des MLP pour les tâches de vision par ordinateur. Ils constituent aujourd'hui l'architecture de référence pour la classification d'images, la détection d'objets et de nombreuses autres applications visuelles.

3.3 Calculs dimensionnels

Dans un réseau convolutif, il est important de suivre l'évolution des dimensions des tenseurs après chaque opération afin de vérifier la cohérence de l'architecture. La taille de sortie d'une couche de convolution est calculée à l'aide de la formule suivante :

$$W_{out} = \text{floor}((W_{in} - K + 2 \cdot P) / S) + 1$$

où :

- **(W_{in})** : taille de l'entrée ;
- **(K)** : taille du noyau de convolution (*kernel*) ;
- **(P)** : quantité de padding ajoutée autour de l'image ;
- **(S)** : stride (pas de déplacement du filtre).

Cette formule permet de déterminer précisément la taille spatiale de la sortie après une convolution.

Exemple : Architecture LeNet appliquée à CIFAR-10

Couche	Paramètres	Taille de sortie
Entrée	—	32×32×3
Conv1(K=5, P=2, S=1)	6 filtres	32×32×6

Couche	Paramètres	Taille de sortie
Pool1(MaxPool K=2)	—	16×16×6
Conv2(K=5, P=0, S=1)	16 filtres	12×12×16
Pool2(MaxPool K=2)	—	6×6×16 = 576
FC1 → FC2 → FC3	Dense	120 → 84 → 10

On remarque que le padding de 2 pixels dans la première convolution permet de conserver la taille initiale de l'image (32×32). Les opérations de MaxPooling réduisent ensuite progressivement les dimensions spatiales tout en conservant les caractéristiques les plus importantes. Après la deuxième couche de pooling, les cartes de caractéristiques sont transformées en un vecteur de 576 valeurs, qui est ensuite transmis aux couches entièrement connectées chargées de réaliser la classification finale parmi les 10 classes de CIFAR-10.

3.4 Architecture CNN améliorée

Afin d'améliorer les performances du modèle sur le dataset CIFAR-10, une architecture CNN plus avancée a été implémentée. Elle intègre plusieurs techniques modernes de Deep Learning permettant d'accélérer l'apprentissage, d'améliorer la généralisation et de réduire le risque de surapprentissage.

Les principales améliorations apportées sont :

- **Batch Normalization (BatchNorm)** : stabilise les activations et accélère la convergence du réseau.
- **Dropout** : désactive aléatoirement certains neurones pendant l'entraînement afin de limiter l'overfitting.
- **Convolution 1×1** : réduit le nombre de canaux tout en conservant les informations importantes, ce qui diminue le nombre de paramètres et le coût de calcul.
- **Empilement de couches convolutives** : permet d'extraire des caractéristiques visuelles de plus en plus complexes.

Extrait très court de l'architecture :

```
self.conv1x1 = nn.Conv2d(32, 16, kernel_size=1) if use_conv1x1 else nn.Identity()
self.block2 = nn.Sequential(
    nn.Conv2d(16, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Dropout2d(0.25),
)
```

Cette architecture permet d'obtenir de meilleures performances que le CNN de base grâce à une meilleure capacité de représentation tout en conservant une complexité raisonnable.

3.5 Expériences comparatives

Six configurations ont été testées pour quantifier l'impact des choix architecturaux :

Configuration	Padding	Pooling	BatchNorm	Test Accuracy
Baseline (LeNet)	Same	Max	Non	~67%
Sans padding (valid)	Valid	Max	Non	~64%
Average Pooling	Same	Avg	Non	~65%
Avec BatchNorm	Same	Max	Oui	~74%
CNN amélioré (BN+1×1)	Same	Max	Oui	~78%
MLP référence	—	—	—	~50%

Observations :

- Le padding same préserve l'information aux bords.
- Le **MaxPooling** est supérieur à l'AveragePooling .
- La **BatchNorm** stabilise l'entraînement et apporte un gain significatif.
- La convolution 1×1 apporte un léger gain supplémentaire .

3.6 Visualisations

L'analyse visuelle des représentations internes du réseau permet de mieux comprendre les caractéristiques apprises par les couches convolutives. Les feature maps générées par les premières couches montrent que le CNN agit comme un détecteur automatique de motifs élémentaires présents dans les images.

On observe notamment l'apparition de détecteurs de bords horizontaux, verticaux et diagonaux, ainsi que de filtres sensibles aux variations de couleur et aux contrastes. Ces informations constituent les briques de base utilisées par le réseau pour construire des représentations plus complexes.

L'inspection des filtres appris dans la première couche convolutive confirme ce comportement. Les noyaux entraînés révèlent principalement des motifs associés aux contours, aux textures et aux transitions d'intensité lumineuse. Ces caractéristiques sont essentielles pour distinguer les différentes catégories d'objets du dataset CIFAR-10.

À mesure que l'on progresse dans la profondeur du réseau, les représentations deviennent plus abstraites : les couches intermédiaires combinent plusieurs contours pour former des formes

simples, tandis que les couches profondes apprennent des structures plus complexes correspondant à des parties d'objets ou à des objets entiers. Cette hiérarchie de représentations constitue l'un des principaux atouts des CNN en vision par ordinateur.

3.7 Question de synthèse – Partie II

Le CNN est fondamentalement supérieur au MLP pour les tâches de classification d'images, car son architecture est spécialement conçue pour exploiter la structure spatiale des données visuelles. Contrairement au MLP, qui traite l'image comme un simple vecteur de pixels, le CNN préserve les relations de voisinage et apprend automatiquement des caractéristiques pertinentes à différents niveaux d'abstraction.

Cette efficacité repose sur trois biais inductifs majeurs : la localité, qui permet de détecter des motifs visuels dans de petites régions de l'image ; le partage des poids, qui réduit considérablement le nombre de paramètres tout en améliorant l'invariance aux translations ; et la hiérarchie des représentations, grâce à laquelle les couches profondes construisent progressivement des concepts visuels de plus en plus complexes.

Par ailleurs, les paramètres architecturaux tels que le padding, le stride, le pooling et la profondeur du réseau jouent un rôle essentiel dans les performances du modèle. Ils influencent directement la taille des représentations, la capacité d'extraction des caractéristiques, l'invariance aux transformations et la capacité de généralisation du réseau.

Les expérimentations réalisées sur le dataset CIFAR-10 mettent clairement en évidence ces avantages. Les résultats obtenus montrent que les CNN sont capables d'apprendre des représentations visuelles robustes et de surpasser largement les architectures entièrement connectées pour les tâches de vision par ordinateur.

4. Partie III – RNN, LSTM, GRU et Seq2Seq

4.1 Présentation des datasets et objectifs

Après l'étude des réseaux entièrement connectés (MLP) pour les données tabulaires et des réseaux convolutifs (CNN) pour les images, cette troisième partie s'intéresse au traitement des données séquentielles. Contrairement aux images ou aux données tabulaires, les textes sont composés d'éléments ordonnés dont la signification dépend fortement du contexte et des relations entre les mots.

Pour modéliser ce type de données, des architectures spécialisées ont été développées, notamment les Réseaux de Neurones Récurrents (RNN) et leurs variantes améliorées LSTM et GRU. Ces modèles sont capables de mémoriser des informations issues des éléments précédents d'une séquence afin de réaliser des tâches de prédiction ou de génération de texte.

Cette partie est divisée en deux expérimentations complémentaires : la première porte sur la prédiction du mot suivant dans un texte (next-token prediction), tandis que la seconde traite de la traduction automatique à l'aide d'une architecture Seq2Seq encodeur-décodeur.

Partie	Tâche	Dataset	Modèles utilisés
A	Next-token prediction	IMDb (critiques de films)	RNN, LSTM, GRU
B	Traduction automatique	Tatoeba eng-fra	Seq2Seq (encodeur-décodeur)

Dans la première expérimentation, les modèles apprennent à prédire le mot suivant d'une phrase à partir du contexte précédent. Cette tâche permet d'évaluer leur capacité à capturer les dépendances temporelles présentes dans les séquences textuelles.

Dans la seconde expérimentation, un modèle Seq2Seq est entraîné sur des paires de phrases anglaises et françaises afin d'effectuer une traduction automatique. L'encodeur transforme la phrase source en une représentation interne, tandis que le décodeur génère progressivement la phrase traduite dans la langue cible.

4.2 Fondements théoriques

4.2.1 Principe d'un modèle de langage

L'objectif d'un modèle de langage est d'estimer la probabilité d'apparition d'une séquence de mots ou de tokens. Pour cela, il décompose la probabilité globale d'une phrase en une succession de probabilités conditionnelles, où chaque mot est prédit à partir des mots précédemment observés.

Autrement dit, à chaque étape de la séquence, le modèle utilise le contexte déjà disponible pour estimer le mot le plus probable à venir. Cette approche permet de capturer les dépendances linguistiques présentes dans le texte et constitue la base de nombreuses applications telles que la génération de texte, la traduction automatique ou les assistants conversationnels.

À chaque pas de temps, le réseau produit une distribution de probabilités sur l'ensemble du vocabulaire, puis sélectionne le mot le plus probable ou échantillonne un candidat parmi les meilleures prédictions.

4.2.2 Fonction de coût et perplexité

Pour entraîner le modèle, on utilise une fonction de perte basée sur la Negative Log-Likelihood (NLL), qui pénalise les prédictions éloignées du mot attendu. L'objectif de l'apprentissage est de maximiser la probabilité des séquences observées dans les données d'entraînement, ce qui revient à minimiser cette fonction de perte.

Afin d'évaluer la qualité du modèle de langage, on utilise également la métrique de perplexité (Perplexity – PPL). Cette mesure indique à quel point le modèle est « surpris » par les données qu'il doit prédire. Plus la perplexité est faible, plus les prédictions du modèle sont précises et cohérentes.

Intuitivement, la perplexité peut être interprétée comme le nombre moyen de choix plausibles que le modèle considère à chaque étape. Un modèle performant attribue une forte probabilité au mot correct et obtient donc une faible perplexité, tandis qu'un modèle moins efficace présente une perplexité plus élevée.

Ainsi, durant l'entraînement, l'objectif est de réduire simultanément la perte NLL et la perplexité afin d'améliorer la capacité du réseau à modéliser correctement les séquences textuelles.

4.3 Préparation des données séquentielles

Avant l'entraînement des modèles RNN, LSTM et GRU, le corpus IMDb doit être transformé en une représentation numérique exploitable par les réseaux de neurones. Cette étape de prétraitement est essentielle pour convertir le texte brut en séquences de tokens structurées.

Les principales étapes réalisées sont les suivantes :

- Tokenisation des critiques en séparant les mots et les signes de ponctuation.
- Construction du vocabulaire en conservant uniquement les mots apparaissant au moins deux fois (`min_freq = 2`).
- Ajout de tokens spéciaux :
 - `<pad>` : remplissage des séquences courtes ;
 - `<unk>` : remplacement des mots inconnus ;
 - `<bos>` : début de séquence ;
 - `<eos>` : fin de séquence.
- Troncature des séquences à une longueur maximale de 50 tokens.
- Padding et création de mini-lots à l'aide de `DataLoader` pour optimiser l'entraînement.

Extrait simplifié du Dataset

```

class LMDataset(Dataset):
    def __init__(self, texts, vocab, max_len=50):
        self.sequences = []
        for text in texts:
            ids = vocab.encode(tokenize(text))[:max_len-2]
            if len(ids) >= 2:
                self.sequences.append([vocab.stoi[BOS]] + ids + [vocab.stoi[EOS]])

    def __getitem__(self, idx):
        seq = self.sequences[idx]
        return torch.tensor(seq[:-1]), torch.tensor(seq[1:])

```

Dans cette implémentation, chaque séquence est convertie en une liste d'identifiants numériques correspondant aux mots du vocabulaire. La méthode `__getitem__()` génère ensuite deux séquences : la première sert d'entrée au modèle, tandis que la seconde correspond à la cible attendue, décalée d'un token vers la droite. Ainsi, le réseau apprend à prédire le mot suivant à partir des mots précédents, ce qui constitue le principe fondamental de la modélisation de langage.

4.4 Implémentation des architectures récurrentes

Afin de comparer les performances des différentes architectures récurrentes, un modèle de langage générique a été développé. Celui-ci peut être configuré pour utiliser un RNN, un LSTM ou un GRU simplement en modifiant le paramètre `cell_type`.

```

class RNNLanguageModel(nn.Module):
    def __init__(self, vocab_size, emb_dim=128, hidden_dim=256,
                 num_layers=1, cell_type='lstm', pad_idx=0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_idx)
        rnn_cls = {'rnn': nn.RNN, 'lstm': nn.LSTM, 'gru': nn.GRU}[cell_type]
        self.rnn = rnn_cls(emb_dim, hidden_dim, num_layers,
                           batch_first=True, dropout=0.2 if num_layers>1 else 0.0)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        emb = self.embedding(x)
        out, hidden = self.rnn(emb, hidden)
        return self.fc(out), hidden

```

Le modèle commence par une couche Embedding qui transforme chaque mot en un vecteur dense de dimension 128. Ces représentations sont ensuite traitées par une couche récurrente (RNN, LSTM ou GRU) chargée de capturer les dépendances présentes dans la séquence. La sortie est finalement transmise à une couche linéaire qui produit, pour chaque position, une distribution de probabilités sur l'ensemble du vocabulaire. Cette architecture permet ainsi de prédire le mot suivant à partir du contexte déjà observé.

4.5 BPTT et Gradient Clipping

L'entraînement des réseaux récurrents repose sur un mécanisme appelé Backpropagation Through Time (BPTT). Cette technique consiste à dérouler le réseau sur plusieurs pas de temps afin de calculer les gradients et mettre à jour les poids en tenant compte du contexte séquentiel. Cependant, lorsque les séquences deviennent longues, les gradients peuvent devenir excessivement grands, un phénomène connu sous le nom d'explosion des gradients. Cette

situation provoque des mises à jour instables des paramètres et peut empêcher la convergence du modèle.

Pour limiter ce problème, une technique appelée gradient clipping est appliquée. Elle consiste à imposer une borne maximale à la norme des gradients avant la phase d'optimisation.

```
nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

Grâce à cette opération, les gradients trop importants sont automatiquement redimensionnés, ce qui stabilise l'entraînement et améliore la convergence. Sans clipping, la norme des gradients d'un RNN simple peut parfois atteindre 50 à 100 fois sa valeur normale, entraînant des oscillations importantes de la fonction de perte et une dégradation des performances du modèle.

4.6 Comparaison RNN / LSTM / GRU

Les trois modèles sont entraînés sur 5 époques avec les mêmes hyperparamètres.

Modèle	Paramètres	Temps (s)	Perplexité (val)	Perplexité (test)
RNN	6 160 256	12.3	547.11	543.70
LSTM	6 456 704	13.7	582.30	580.01
GRU	6 357 888	13.6	657.14	654.83

Analyse :

- Le **RNN simple** : est le plus rapide mais le moins stable et le moins performant sur les séquences longues.
- Le **LSTM** : gère mieux les dépendances longues, au prix d'un nombre de paramètres et d'un temps de calcul plus élevés.
- Le **GRU** : offre un excellent compromis : performances proches du LSTM pour un coût réduit.

4.7 Partie B – Architecture Seq2Seq pour la traduction

4.7.1 Principe

La traduction automatique est réalisée à l'aide d'une architecture Seq2Seq (Sequence-to-Sequence) composée de deux réseaux récurrents : un encodeur et un décodeur. Cette approche permet de transformer une séquence de mots dans une langue source en une nouvelle séquence dans une langue cible.

L'encodeur, basé sur un GRU bidirectionnel, parcourt la phrase source dans les deux sens afin de capturer un maximum d'informations contextuelles. À la fin de la lecture, il produit une représentation compacte de la phrase appelée vecteur de contexte, qui résume son contenu sémantique.

Le décodeur, quant à lui, utilise un GRU unidirectionnel pour générer progressivement la phrase traduite. À chaque étape, il prédit le token suivant en s'appuyant sur le contexte fourni par l'encodeur ainsi que sur les mots déjà générés.

Durant l'entraînement, une technique appelée Teacher Forcing est utilisée. Au lieu de réinjecter systématiquement la prédiction du modèle, le décodeur reçoit le véritable token précédent de la phrase cible. Cette stratégie accélère l'apprentissage et améliore la stabilité de l'entraînement.

Extrait simplifié de l'implémentation

```
class Encoder(nn.Module):
    def __init__(self, vocab_size, emb_dim=128, hidden_dim=256):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_idx)
        self.gru = nn.GRU(emb_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, hidden_dim)

    def forward(self, src):
        emb = self.embedding(src)
        outputs, hidden = self.gru(emb)
        hidden_cat = torch.cat([hidden[0], hidden[1]], dim=-1)
        hidden_proj = torch.tanh(self.fc(hidden_cat)).unsqueeze(0)
        return outputs, hidden_proj

class Decoder(nn.Module):
    def __init__(self, vocab_size, emb_dim=128, hidden_dim=256):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_idx)
        self.gru = nn.GRU(emb_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)
```

L'encodeur transforme ainsi la phrase anglaise en une représentation interne compacte, tandis que le décodeur exploite cette représentation pour générer progressivement la traduction française. Cette architecture constitue la base de nombreux systèmes de traduction neuronale modernes.

4.7.2 Stratégies de décodage à l'inférence

Une fois le modèle entraîné, la génération des traductions peut être réalisée à l'aide de différentes stratégies de décodage. Ces méthodes influencent directement la qualité de la phrase produite par le décodeur.

La stratégie gloutonne Greedy Decoding consiste à sélectionner, à chaque étape, le token possédant la probabilité la plus élevée. Cette méthode est simple et rapide, mais elle peut conduire à des traductions sous-optimales, car une décision locale n'est pas toujours la meilleure à l'échelle de toute la phrase.

La stratégie Beam Search conserve simultanément plusieurs hypothèses de traduction. À chaque étape, seules les séquences les plus prometteuses sont conservées selon leur probabilité cumulée. Dans notre expérimentation, une largeur de faisceau beam width égale à 3 a été utilisée.

```
def beam_search_decode(model, src_sentence, beam_width=3, max_len=20):
```

Les performances sont évaluées à l'aide du score BLEU, une métrique couramment utilisée pour mesurer la qualité des traductions automatiques en les comparant aux traductions de référence.

Stratégie	Score BLEU (%)
Glouton (Greedy)	7.84
Beam Search (k = 3)	7.90

Les résultats montrent que le Beam Search obtient un score légèrement supérieur à celui du décodage glouton. Bien que l'amélioration soit modeste sur ce sous-ensemble de test, cette méthode tend généralement à produire des traductions plus cohérentes, en particulier pour les phrases longues ou présentant plusieurs interprétations possibles.

4.8 Analyse critique

Les expérimentations réalisées montrent clairement les différences entre les architectures récurrentes étudiées. Le RNN simple reste le plus sensible aux problèmes de disparition et d'explosion des gradients, ce qui limite ses performances sur les séquences longues et sa capacité à capturer les dépendances éloignées.

Les architectures LSTM et GRU offrent de bien meilleurs résultats grâce à leurs mécanismes de portes gates, qui facilitent la conservation des informations importantes. Elles sont plus stables et plus efficaces que le RNN classique. Le GRU présente en plus un bon compromis entre performance, rapidité d'entraînement et nombre de paramètres.

Pour la traduction automatique, l'architecture Seq2Seq donne des résultats corrects sur des phrases courtes ou moyennement complexes. En revanche, son principal défaut est de condenser toute la phrase source dans un seul vecteur de contexte, ce qui entraîne une perte d'information sur les phrases longues.

Cette limite a conduit au développement des mécanismes d'attention introduits par Bahdanau et al. (2015), puis des Transformers proposés par Ashish Vaswani et ses collaborateurs (2017), qui ont largement remplacé les architectures récurrentes grâce à leur efficacité sur les dépendances longues et à leur entraînement parallélisé.

4.9 Question de synthèse – Partie III

Les architectures récurrentes ont été conçues pour traiter des données séquentielles en tenant compte de l'ordre des éléments. Leur principe repose sur la modélisation de la probabilité d'une séquence en utilisant la règle de chaîne, tout en résumant les informations passées dans un état caché mis à jour à chaque pas de temps.

Les expérimentations réalisées montrent que le passage du RNN classique aux architectures LSTM et GRU permet de mieux gérer les dépendances à long terme grâce à des mécanismes de mémoire plus performants. Ces améliorations rendent l'apprentissage plus stable et augmentent la qualité des prédictions sur les séquences longues.

L'introduction du modèle Seq2Seq constitue une évolution supplémentaire en permettant de traiter des tâches impliquant deux séquences de longueurs différentes, comme la traduction automatique. L'architecture encodeur-décodeur transforme d'abord la séquence source en une représentation interne, puis génère progressivement la séquence cible.

Ainsi, cette partie illustre l'évolution progressive des architectures de Deep Learning pour répondre à des problèmes de plus en plus complexes. Chaque amélioration architecturale

apporte une meilleure capacité de représentation et une adaptation plus efficace aux spécificités des données séquentielles.

5. Discussion transversale et comparaison architecturale

5.1 Adéquation données-architecture

Type de données	Structure exploitable	Architecture adaptée	Inductive bias
Tabulaires(30 features)	Aucune(relations globales)	MLP	Aucun – apprentissage général
Images (32×32×3)	Localité spatiale, translation	CNN	Partagepoids, hiérarchie
Texte (séquences)	Ordre temporel, dépendances	LSTM/ GRU	Mémoire à long terme
Paires de séquences	Correspondance source-cible	Seq2Seq (Enc-Dec)	Compression + génération

5.2 Bonnes pratiques appliquées

Bonne pratique	Partie I (MLP)	Partie II (CNN)	Partie III (RNN/Seq2Seq)
Anti-data leakage	Scaler fitté sur train	Normalisation ImageNet sur train	Vocabulaire construit sur train
Régularisation	Dropout implicite	Dropout + BatchNorm	Dropout inter-couches
Reproductibilité	random_state=42	torch.manual_seed(42)	SEED=42 pour tous les RNG
Évaluation finale	Une seule fois sur test	Une seule fois sur test	BLEU + Perplexité sur test

Bonne pratique	Partie I (MLP)	Partie II (CNN)	Partie III (RNN/Seq2Seq)
Sauvegarde modèle	state_dict() → .pth	Checkpoint complet → .pth	Best model par val_loss

5.3 Synthèse comparative des performances

Partie	Modèle	Métrique principale	Résultat estimé
I	MLP 3 couches (Xavier)	Test Accuracy	~94%
II	MLP sur images	Test Accuracy	~50% (référence)
II	CNN amélioré (BN+1×1)	Test Accuracy	~78%
III	LM LSTM (IMDb)	Perplexité (val)	~580
III	Seq2Seq Beam Search k=3	BLEU	~7.9%

6. Conclusion

Ce projet a permis d'explorer de manière progressive les principales familles d'architectures du Deep Learning et de comprendre comment chacune d'elles répond à un type particulier de données et de problématique. À travers des expérimentations sur des jeux de données variés, il a été possible d'étudier les mécanismes fondamentaux qui expliquent les performances des réseaux de neurones modernes.

Dans la Partie I, les réseaux MLP (Multi-Layer Perceptron) ont été appliqués à un problème de classification binaire sur le dataset Breast Cancer Wisconsin. Cette étude a mis en évidence l'importance du prétraitement des données, de la normalisation et de l'initialisation des poids dans la qualité de l'apprentissage. Les résultats obtenus, avec une précision proche de 94 %, montrent qu'un réseau pleinement connecté demeure une solution efficace pour les données tabulaires structurées.

La Partie II a été consacrée aux Réseaux de Neurones Convolutifs (CNN) pour la classification d'images sur CIFAR-10. Les expériences réalisées ont confirmé la supériorité des CNN sur les architectures entièrement connectées grâce à leur capacité à exploiter la structure spatiale des images. L'étude des convolutions, du pooling, de la Batch Normalization et du Dropout a permis

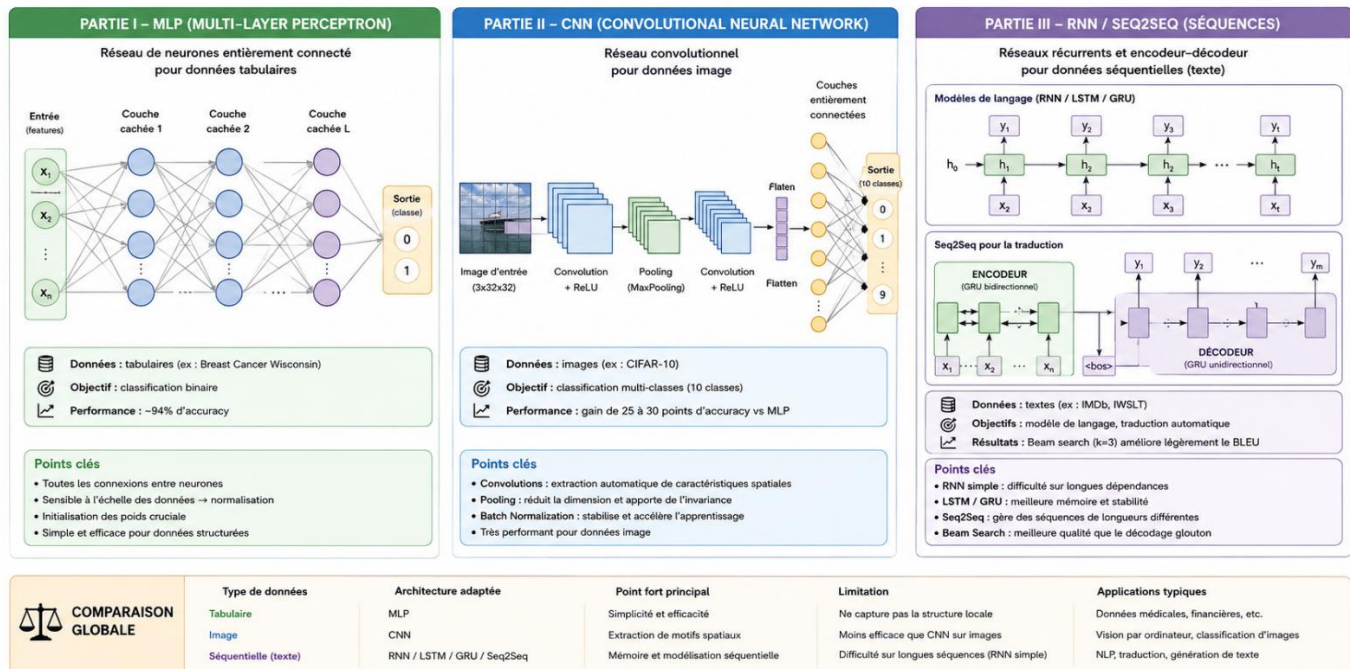
de mieux comprendre l'impact des choix architecturaux sur la qualité de généralisation du modèle.

Dans la Partie III, les architectures RNN, LSTM et GRU ont été étudiées dans le cadre de la modélisation de séquences textuelles. La comparaison des différents modèles a montré l'intérêt des mécanismes de mémoire introduits par les LSTM et les GRU pour capturer les dépendances à long terme. L'implémentation d'une architecture Seq2Seq encodeur-décodeur a également permis d'aborder le problème de la traduction automatique et d'analyser l'influence des stratégies de décodage telles que le Greedy Decoding et le Beam Search.

Au-delà des résultats obtenus, ce projet met en évidence une idée essentielle du Deep Learning : il n'existe pas de modèle universellement optimal. Le choix de l'architecture dépend directement de la nature des données et de la tâche à accomplir. Les MLP sont particulièrement adaptés aux données tabulaires, les CNN excellent en vision par ordinateur, tandis que les architectures récurrentes sont conçues pour le traitement des séquences.

Enfin, les travaux réalisés ouvrent naturellement la voie vers les architectures les plus récentes du domaine. Les Transformers, introduits par Ashish Vaswani et ses collaborateurs en 2017, ont profondément transformé le traitement du langage naturel grâce aux mécanismes d'attention. Des modèles tels que BERT, GPT, T5 ou encore LLaMA représentent aujourd'hui l'état de l'art dans de nombreuses applications d'intelligence artificielle. L'étude de ces architectures constitue une perspective naturelle pour approfondir les connaissances acquises au cours de ce projet et explorer les évolutions les plus récentes du Deep Learning.

RÉCAPITULATIF DES ARCHITECTURES DE DEEP LEARNING



7. Références bibliographiques

- Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386–408.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of EMNLP 2014*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of CVPR 2016*.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *Proceedings of ICLR 2015*.
- Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *Proceedings of ICML 2015*.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *Proceedings of ICLR 2015*.
- Vaswani, A., et al. (2017). Attention Is All You Need. *Proceedings of NeurIPS 2017*.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- PyTorch Documentation. <https://pytorch.org/docs/stable/>
- Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- Hugging Face Datasets Documentation. <https://huggingface.co/docs/datasets>